

Stereo-Based Tire Vibration Measurement

Code Manual

This manual explains how to use the code files in the stereo-based tire vibration measurement project. For each script, it describes its purpose, required inputs, configuration parameters, outputs, and position in the processing pipeline.

1. Project Objective

The objective is to measure tire vibration using two synchronized cameras and identify dominant natural frequencies and vibration mode shapes.

The complete workflow is:

1. Calibrate the two cameras.
2. Record synchronized images after impacting the tire.
3. Define the tire region and extract tire edges.
4. Detect bright reflective studs.
5. Track the detected studs in both camera views.
6. Match corresponding points and reconstruct their 3D trajectories.
7. Analyze the reconstructed trajectories in the frequency domain.
8. Extract and visualize vibration mode shapes.
9. Optionally run edge-based comparison methods.

The recommended primary method is blob-based stud tracking. The edge-based methods are mainly included for comparison and discussion.

2. Installation

Python 3.10 or newer is recommended.

Install the main dependencies with:

```
pip install numpy pandas scipy matplotlib opencv-python
```

Or use the project requirements file:

```
pip install -r requirements.txt
```

The `acquisition.py` script also requires the Hikrobot camera SDK. The SDK usually needs to be available in a folder such as:

```
MvImport/  
├─MvCameraControl_class.py  
└─CameraParams_header.py
```

3. Input and Output File Formats

ROI edges

```
Frame_Name,Edge_Type,Point_ID,X,Y
```

Blob detections

```
Frame_Name,Blob_ID,X,Y,Diameter
```

2D tracks

```
Frame_Name,Camera,Track_ID,X,Y
```

3D trajectories

```
Frame_ID,Stud_ID,X,Y,Z
```

The column names must remain consistent between processing stages.

Phase 0 — Stereo Calibration

4. `calibration.py`

Purpose

This script calculates the intrinsic and extrinsic parameters of both cameras from checkerboard image pairs.

The generated calibration parameters include:

```
K1,D1,K2,D2  
R,T  
E,F  
P1, P2
```

Input

Two folders containing corresponding checkerboard images:

```
data/calibration/  
├── camA/  
└── camB/
```

Each image from camera A must have a corresponding image from camera B.

Important settings

Edit the following values in the main() function:

```
folder_a="data/calibration/camA"  
folder_b="data/calibration/camB"  
output_file = "data/calibration/stereo_calibration.npz"
```

The checkerboard dimensions are the number of inner corners, not the number of squares:

```
pattern_cols=13  
pattern_rows=20  
square_size_mm=19.0
```

Run

```
python code/calibration.py
```

Output

```
data/calibration/stereo_calibration.npz
```

Before continuing, check the number of valid image pairs and the calibration RMS errors.

Phase 1 — Image Acquisition

5. acquisition.py

Purpose

This script configures a Hikrobot camera for external triggering and saves the frames received after Arduino trigger pulses.

Hardware requirements

- Hikrobot camera.
- Arduino or external trigger source.
- Hikrobot SDK.
- GigE or USB camera connection.

Important settings

```
CAMERA_LABEL="CAM_A"  
OUTPUT_ROOT=Path("data/raw")  
EXPOSURE_US=3000.0  
FRAME_TIMEOUT_MS=2000  
VIDEO_FPS = 80.0
```

For the second camera computer, use:

```
CAMERA_LABEL = "CAM_B"
```

Run

```
python code/acquisition.py
```

Output

```
data/raw/CAM_A_YYYYMMDD_HHMMSS/  
├──output.avi  
└──timestamps.csv
```

The timestamp file contains:

```
frame_idx,timestamp_ns
```

This script depends on the real camera hardware and SDK version and must be tested on the camera computer.

Phase 2 — ROI and Edge Extraction

6. roi_edges.py

Purpose

This script detects the tire region and saves the outer and inner contours for each image. These contours are later used to create the blob-detection mask and in the edge-based methods.

Input

Set the image folder:

```
INPUT_DIR = Path("data/raw/selected_pairs")
```

Output

```
OUTPUT_CSV = Path("data/intermediate/roi_edges.csv")
```

Output format:

```
Frame_Name,Edge_Type,Point_ID,X,Y
```

Run

```
python code/roi_edges.py
```

Controls

S	Save the contours for the current frame
A	Automatically save the remaining frames
Space	Skip the current frame
Q	Quit

If the two cameras are processed separately, create separate output files:

```
roi_edges_camA.csv  
roi_edges_camB.csv
```

Phase 3 — Stud Detection and 2D Tracking

7. blob_detection.py

Purpose

This script detects bright reflective studs inside the ROI. It uses OpenCV's SimpleBlobDetector.

Input

- Camera images.
- ROI edge CSV file.

Set the paths with:

```
INPUT_DIR=Path("data/raw/selected_pairs")
ROI_CSV_FILENAME=Path("data/intermediate/roi_edges.csv")
BLOB_OUTPUT_CSV = Path("data/intermediate/detections.csv")
```

Important parameters

Separate profiles are provided for the two cameras:

```
PARAMS={
    "camA": {...},
    "camB": {...},
}
```

The main parameters are:

- `threshold`: intensity threshold.
- `min_area`: minimum blob area.
- `max_area`: maximum blob area.
- `min_circularity`: minimum circularity.
- `min_convexity`: minimum convexity.
- `min_inertia`: minimum inertia ratio.

Run

```
python code/blob_detection.py
```

Controls

S	Save detections for the current frame
A	Automatically process the remaining frames
Space	Skip the current frame
Q	Quit

Output

```
Frame_Name,Blob_ID,X,Y,Diameter
```

If the cameras are processed separately, keep separate outputs:

```
detections_camA.csv
detections_camB.csv
```

8. tracking.py

Purpose

This script converts frame-by-frame blob detections into continuous tracks.

The tracking method uses:

- A constant-velocity Kalman filter.
- The Hungarian algorithm for assignment.
- A maximum allowed displacement between frames.
- Removal of short or unreliable tracks.

Input

```
data/intermediate/detections.csv
```

Important settings

```
MIN_LIFESPAN_PERCENT=0.20
```

```
MAX_SKIPPED_FRAMES = 10
```

Camera A and camera B parameters are configured separately.

Run

```
python code/tracking.py
```

Output

```
data/intermediate/cleaned_tracks.csv
```

Output format:

```
Frame_Name,Camera,Track_ID,X,Y
```

9. view_2d_tracks.py

Purpose

This script provides a visual inspection tool for the 2D stud trajectories. It is a validation tool rather than a core processing step.

Input

- Camera A and camera B images.
- cleaned_tracks.csv.

Run

python code/view_2d_tracks.py

Controls

T	Toggle camera
Space	Play or pause
Q	Quit

What to check

- Points should remain on the real reflective studs.
- Track identities should not switch between studs.
- Trajectories should not contain unrealistic jumps.
- There should not be too many short tracks.

Phase 4 — Stereo Matching and 3D Reconstruction

10. stereo_reconstruction.py

Purpose

This script matches tracks from camera A and camera B using epipolar geometry and converts matched points into 3D coordinates.

Input

```
TRACKS_CSV=Path("data/intermediate/cleaned_tracks.csv")
CALIB_NPZ = Path("data/calibration/stereo_calibration.npz")
```

Internal steps

1. Load the tracks.

2. Group synchronized frames.
3. Calculate epipolar distances.
4. Perform one-to-one assignment using the Hungarian algorithm.
5. Use consensus voting to establish persistent track correspondences.
6. Triangulate matched points with `cv2.triangulatePoints`.

Important parameters

```
EPIPOLAR_THRESHOLD=15.0  
MIN_MATCH_FRACTION=0.05  
Y_SHIFT = 0.0
```

Run

```
python code/stereo_reconstruction.py
```

Outputs

```
data/processed/stereo_matches_2d.csv  
data/processed/final_3d_paths.csv
```

3D trajectory format:

```
Frame_ID,Stud_ID,X,Y,Z
```

11. view_3d_paths.py

Purpose

This script displays the reconstructed 3D trajectories interactively so that the tire-ring geometry and point motion can be checked.

Input

```
data/processed/final_3d_paths.csv
```

Run

```
python code/view_3d_paths.py
```

What to check

- The points should form the general shape of the tire ring.
- The radius should not change unrealistically between frames.
- Large jumps in X, Y, or Z should be investigated.
- The coordinate unit depends on the calibration square size.

Phase 5 — Frequency Analysis and Mode Shapes

12. frequency_analysis.py

Purpose

This script converts the selected 3D coordinate, usually Y, into a time signal for each stud and calculates spectra and dominant frequencies.

Input

```
data/processed/final_3d_paths.csv
```

Important settings

```
INPUT_FILE=Path("data/processed/final_3d_paths.csv")  
FPS = 75.0
```

FPS must match the actual acquisition frame rate.

Run

```
python code/frequency_analysis.py
```

Interactive features

- Red slider: high-pass frequency.
- Green slider: low-pass frequency.
- Click a stud: show its time signal and FFT spectrum.

Purpose of the result

This step is used to identify dominant frequencies for individual studs and to inspect signal quality.

13. modal_shapes.py

Purpose

This script extracts tire vibration mode shapes from the reconstructed 3D trajectories.

The steps are:

1. Build a time-by-stud matrix.
2. Fill missing samples.
3. Remove the mean displacement of each stud.
4. Calculate the FFT.
5. Detect modal peaks.
6. Extract complex amplitude and phase.
7. Remove the global phase.
8. Build normalized signed mode shapes.

Important settings

```
FPS=75.0
COORDINATE_COLUMN="Y"
MIN_FREQUENCY=0.5
PEAK_RELATIVE_THRESHOLD=0.10
NUMBER_OF_MODES=2
DEFORM_SCALE = 80.0
```

Run

```
python code/modal_shapes.py
```

Outputs

```
data/processed/mode_shapes.csv
data/processed/modal_frequencies.csv
```

DEFORM_SCALE is only used to exaggerate the visualization. It is not the physical vibration amplitude.

Phase 6 — Optional Edge-Based Methods

14. edge_mean_radius.py

Purpose

This script extracts the inner tire contour from both cameras, reconstructs the edge points in 3D, and calculates the mean radius of the ring over time.

Use

This method is included for comparison with the blob-based approach. Circumferential averaging can cancel positive and negative bending-mode lobes, so it is not the primary method of the project.

Important settings

```
PAIRS_FOLDER=Path("data/raw/selected_pairs")
CALIBRATION_FILE=Path("data/calibration/stereo_calibration.npz")
FPS=75.0
POINTS_PER_RING = 360
```

Run

```
python code/edge_mean_radius.py
```

Outputs

```
data/processed/edge_mean_radius_signal.csv
data/processed/edge_mean_radius_spectrum.csv
```

15. edge_single_point.py

Purpose

This script selects one point from the reconstructed edge ring and analyzes one of its coordinates, usually Y, over time.

Important settings

```
PAIRS_FOLDER=Path("data/raw/selected_pairs")
CALIBRATION_FILE=Path("data/calibration/stereo_calibration.npz")
POINT_INDEX=0
COMPONENT="y"
FPS = 75.0
```

To analyze another coordinate:

```
COMPONENT = "x"
```

or:

```
COMPONENT = "z"
```

Run

```
python code/edge_single_point.py
```

Outputs

```
data/processed/edge_single_point_signal.csv  
data/processed/edge_single_point_spectrum.csv
```

Interpretation

A low-frequency peak may be related to global tire motion or depth artifacts. This method should mainly be used as qualitative confirmation of the dominant frequency.

Common Problems

Calibration finds too few valid pairs

- Check the number of inner checkerboard corners.
- Remove blurry images.
- Use checkerboard positions and orientations that cover the image area.

Too many or too few blobs are detected

- Adjust the intensity threshold.
- Tune `min_area`.
- Adjust circularity and convexity.
- Improve the ROI.

Tracks jump between studs

- Reduce `max_jump`.
- Adjust the Kalman process noise.
- Remove short tracks.
- Inspect the result with `view_2d_tracks.py`.

The reconstructed 3D shape is incorrect

- Check synchronization between the two cameras.
- Use a calibration file from the same camera setup.
- Verify the checkerboard square size.
- Reduce the epipolar threshold.

Frequencies are not reasonable

- Set FPS to the actual camera frame rate.
- Use a sufficiently long recording.
- Remove drift and global motion where appropriate.
- Compare the blob-based result with the edge-based results.